

# Generating Structured Outputs

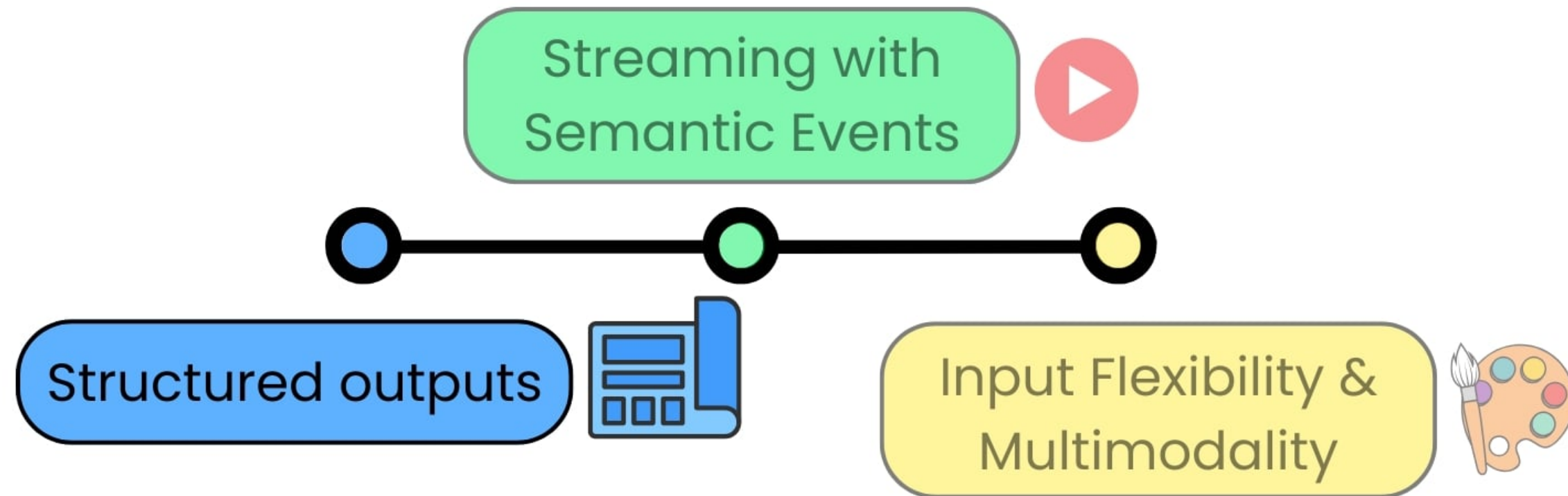
WORKING WITH THE OPENAI RESPONSES API



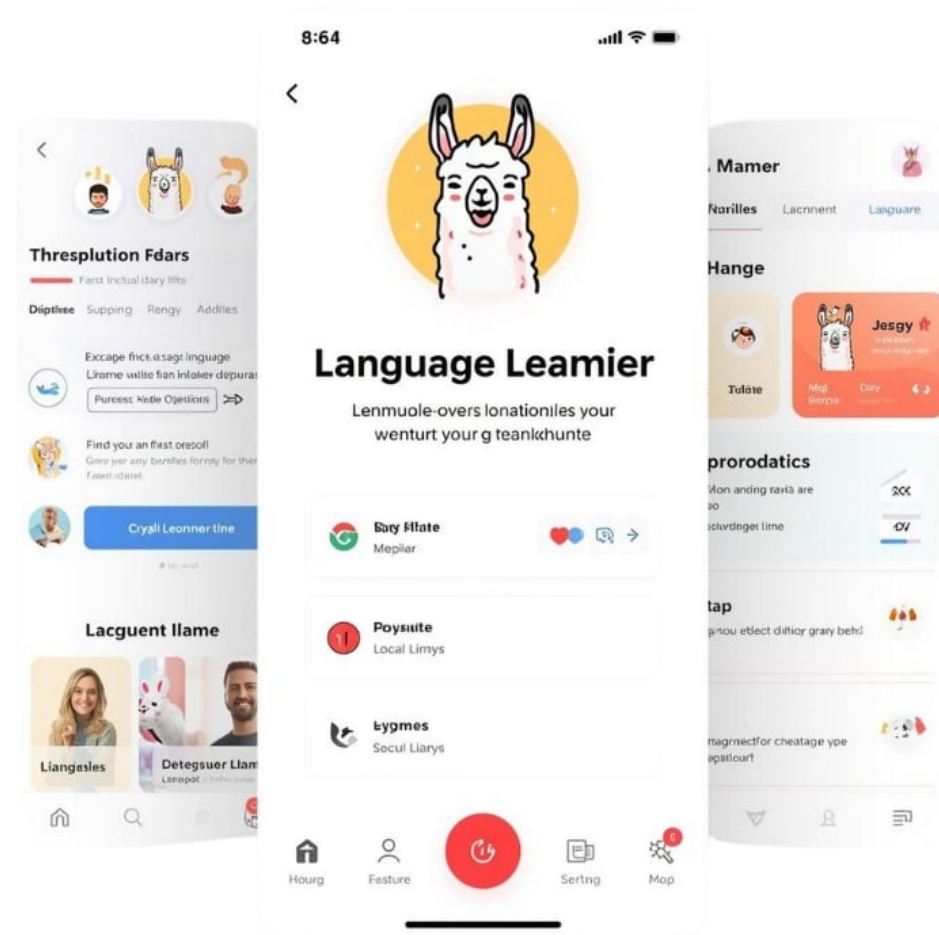
**James Chapman**

AI Curriculum Manager, DataCamp

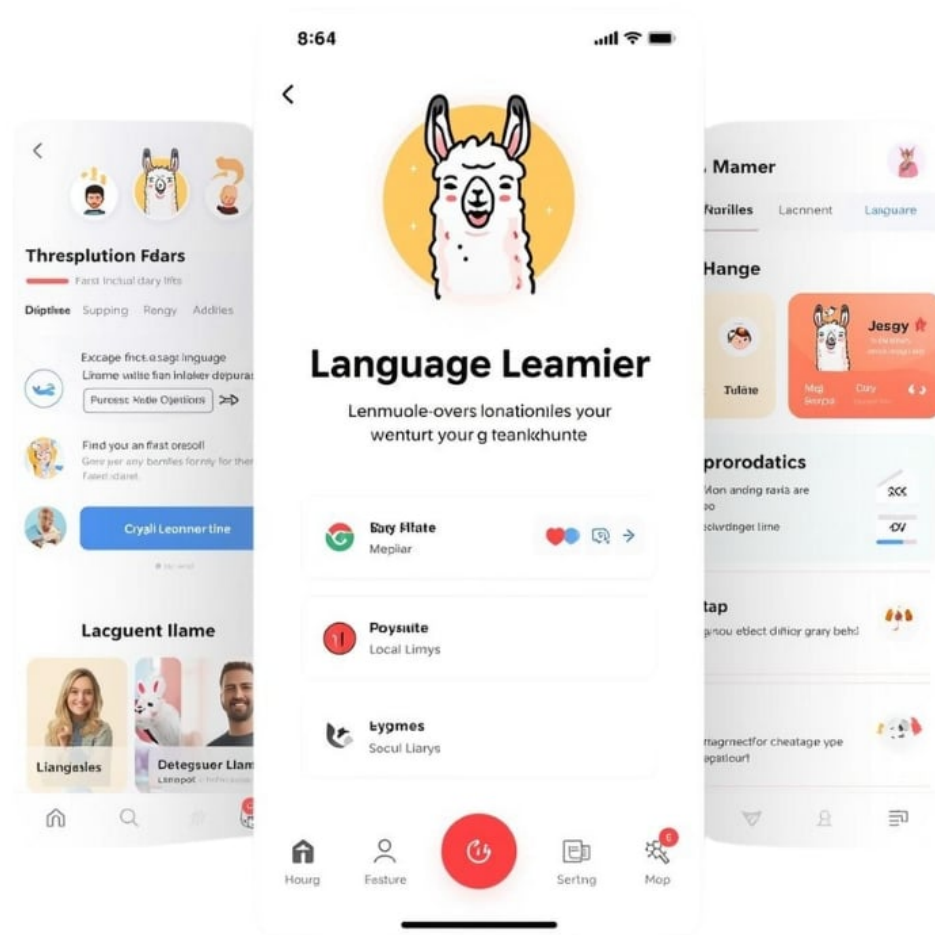
# Coming up...



# The Need for Structure



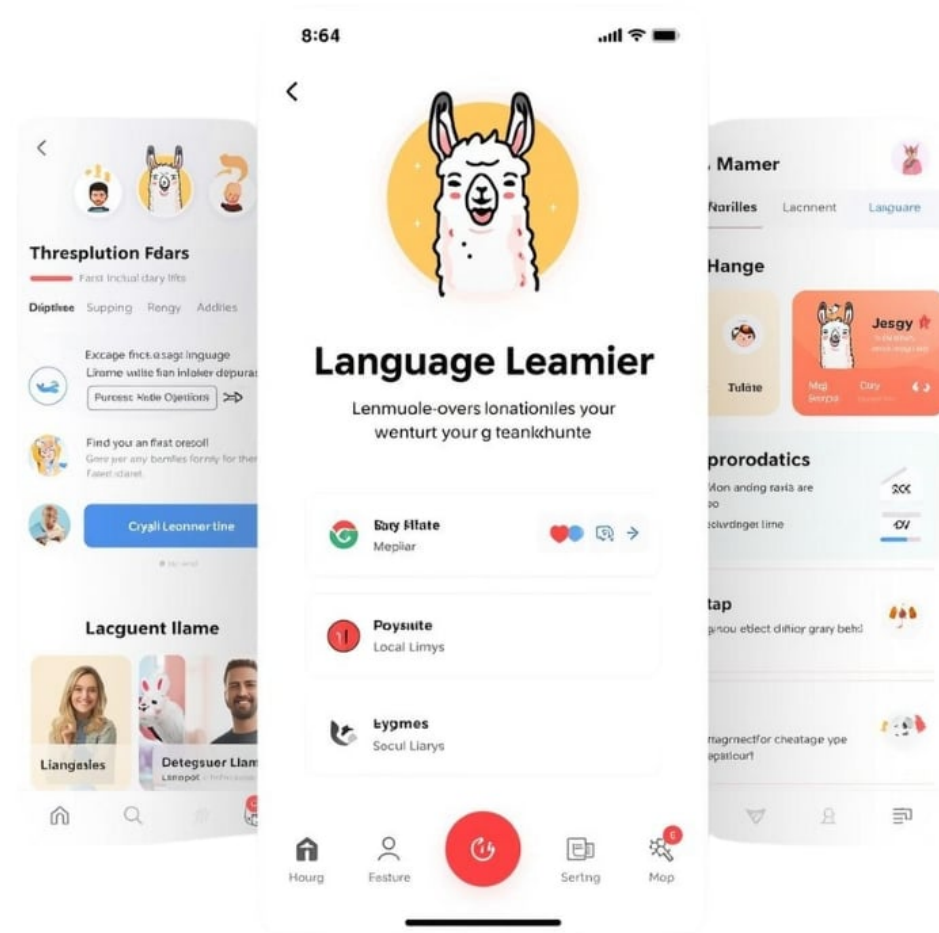
# The Need for Structure



What is this animal?

- ☐ Cane
- ☐ Uccello
- ☒ **Gatto**
- ☐ Pesce

# The Need for Structure



What is this animal?

 Cane

Uccello

 **Gatto**

Pesce

```
{  
  "score": 0  
  "passed": False  
  "feedback": "'Cane' means Dog."  
}
```

# Part 1: Defining the Output Schema

```
from pydantic import BaseModel

class QuizResult(BaseModel):
    score: int
    passed: bool
    feedback: str
```

# Part 1: Defining the Output Schema

```
from pydantic import BaseModel, Field

class QuizResult(BaseModel):
    score: int = Field(description="Number of correct answers out of 10")
    passed: bool = Field(description="True if score is 7 or higher")
    feedback: str = Field(
        description="Encouraging message with specific tips for improvement"
    )
```

## Part 2: Sending the Request

```
response = client.responses.parse(  
    model="gpt-5.4-mini",  
    instructions="You are a Spanish vocabulary tutor. Grade the student's quiz  
answers. Grade the quiz with 2 points per correct answer.",  
    input="""1. casa = house  
           2. perro = dog  
           3. gato = car  
           4. libro = book  
           5. agua = water""",  
    text_format=QuizResult  
)
```

# Part 3: Extracting the Results

```
result = response.output_parsed
print(f"Score: {result.score}/10")
print(f"Passed: {result.passed}")
print(f"Feedback: {result.feedback}")
```

Score: 8/10

Passed: True

Feedback: Great job – you scored 8/10 (4/5 correct). The only mistake was #3: 'gato' means 'cat', not 'car' (Spanish for 'car' is 'coche' or 'carro'). Tip: review common animal vocabulary with flashcards and short quizzes to reinforce recall.

# More Complex Data Structures

```
class Mistake(BaseModel):
    word: str = Field(description="The Spanish word that was incorrect")
    student_answer: str = Field(description="What the student wrote")
    correct_answer: str = Field(description="The correct translation")

class DetailedQuizResult(BaseModel):
    score: int = Field(description="Number of correct answers out of 10")
    passed: bool = Field(description="True if score is 7 or higher")
    feedback: str = Field(description="Encouraging message with specific tips")
    mistakes: list[Mistake] = Field(description="List of incorrect answers")
```

```
response = client.responses.parse(  
    model="gpt-5.4-mini",  
    instructions="You are a Spanish vocabulary tutor. Grade the student's quiz  
answers. Grade the quiz with 2 points per correct answer.",  
    input="""1. casa = house  
            2. perro = dog  
            3. gato = car  
            4. libro = library  
            5. agua = water""",  
    text_format=DetailedQuizResult  
)
```

```
result = response.output_parsed
print(f"Score: {result.score}/10")
print(f"Passed: {result.passed}")
for mistake in result.mistakes:
    print(f"{mistake.word}: '{mistake.student_answer}' -> '{mistake.correct_answer}'")
```

```
Score: 6/10
Passed: False
gato: 'car' -> 'cat'
libro: 'library' -> 'book'
```

# Summary

```
from pydantic import BaseModel, Field

class QuizResult(BaseModel):
    score: int = Field(...)
    passed: bool = Field(...)
    feedback: str = Field(...)
```

```
response = client.responses.parse(
    model="gpt-5.4-mini",
    instructions="...",
    input="...",
    text_format=QuizResult
)
```

```
result = response.output_parsed

print(f"Score: {result.score}/10")
print(f"Passed: {result.passed}")
print(f"Feedback: {result.feedback}")
```

# Let's practice!

WORKING WITH THE OPENAI RESPONSES API

# Streaming with Semantic Events

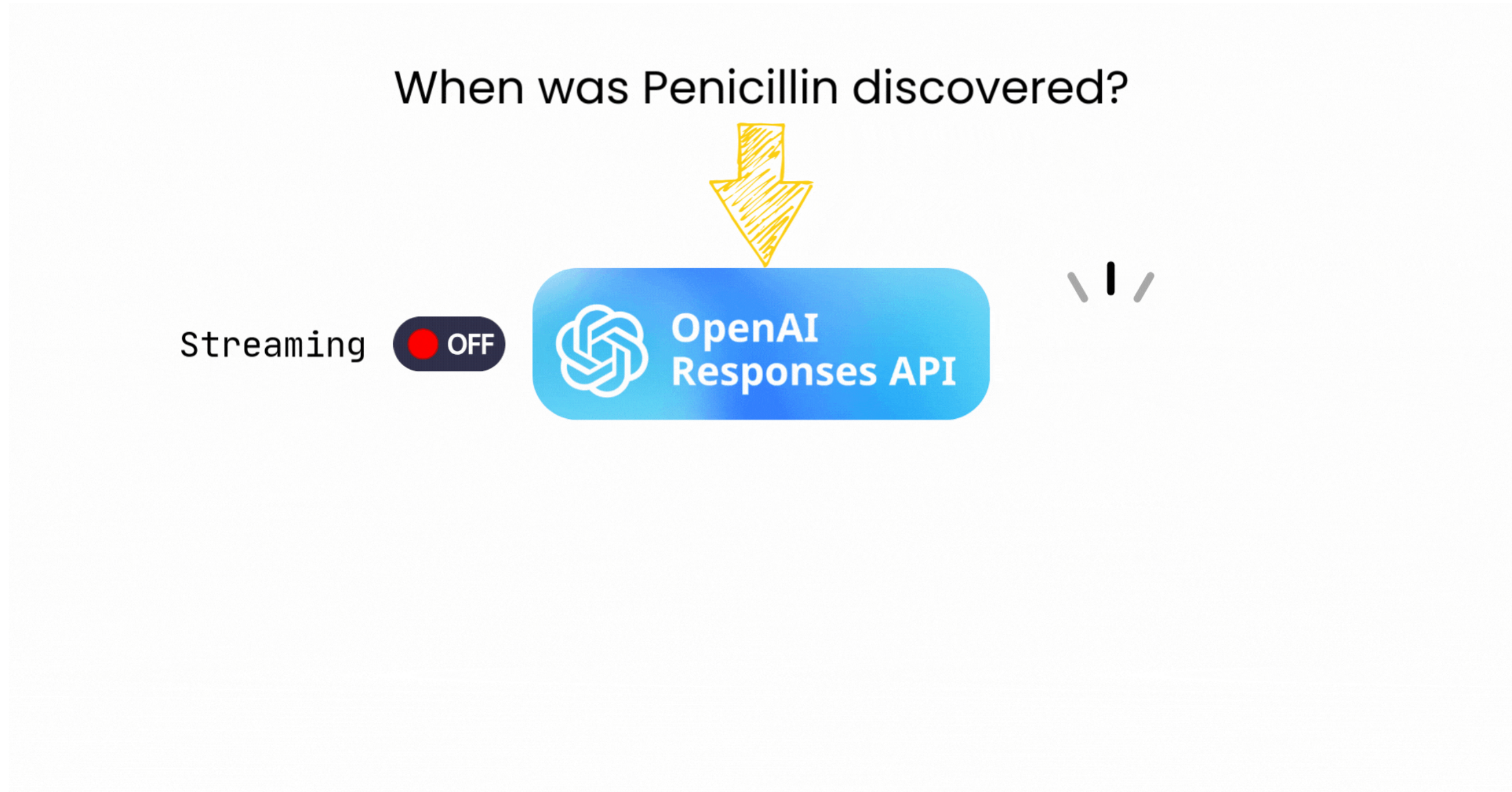
WORKING WITH THE OPENAI RESPONSES API



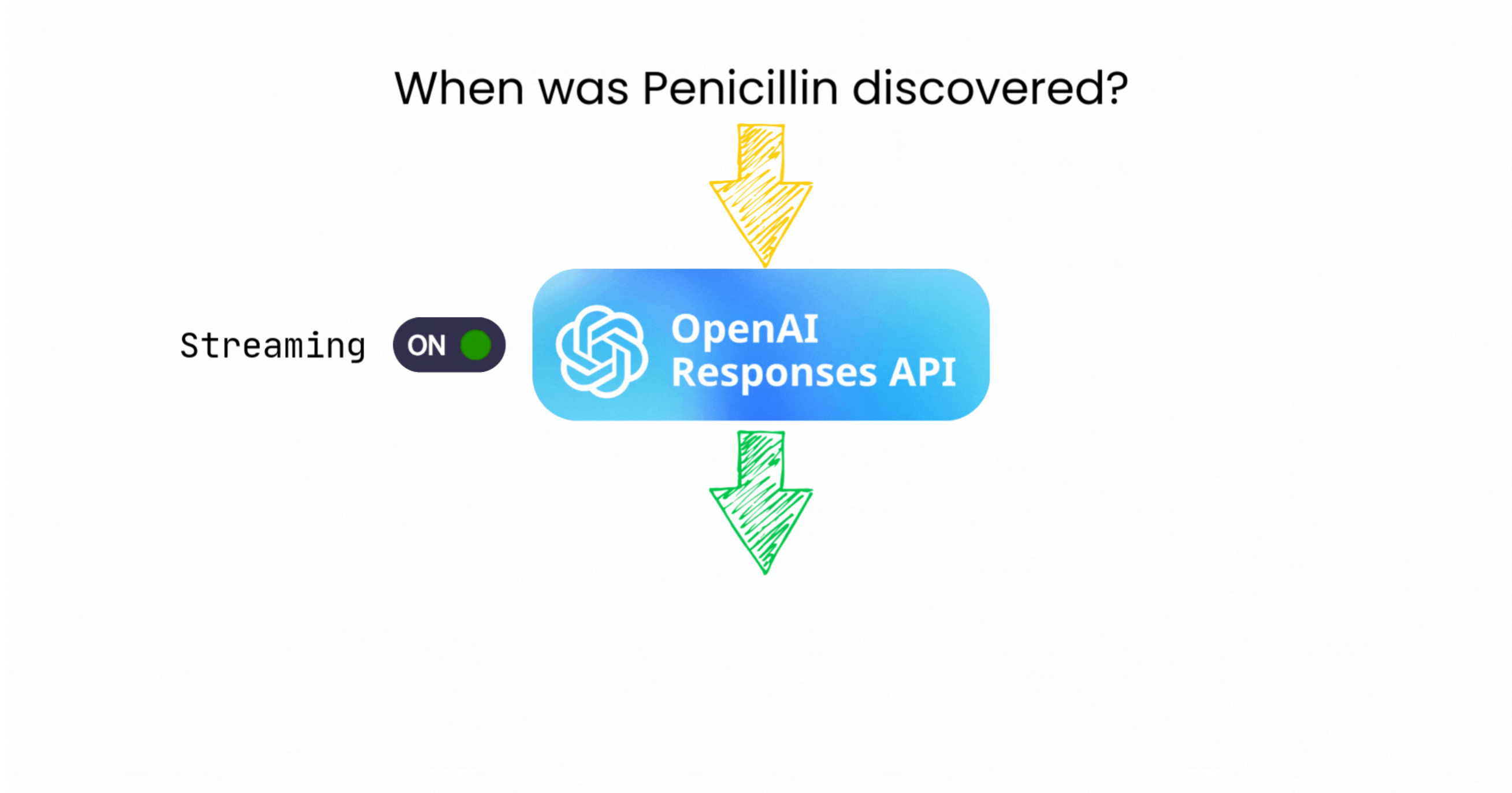
**James Chapman**

AI Curriculum Manager, DataCamp

# Why Stream?



# Why Stream?



# Semantic Events

- Structured updates that describe what's happening

# Semantic Events

- Structured updates that describe what's happening

| Event Type                                          | Description                      |
|-----------------------------------------------------|----------------------------------|
| <code>response.created</code>                       | The model has started generating |
| <code>response.output_text.delta</code>             | Partial text update              |
| <code>response.output_text.done</code>              | Text block complete              |
| <code>response.function_call.arguments.delta</code> | Streaming tool arguments         |
| <code>response.completed</code>                     | The entire response is finished  |

# Example: Basic Text Streaming

```
prompt = "Explain the seasons of the year concisely to a child."

with client.responses.create(model="gpt-5.4-mini", input=prompt, stream=True) as stream:
    current_text = ""

    for event in stream:
        if event.type == "response.output_text.delta":
            current_text += event.delta
            print(current_text)
```

# Example: Basic Text Streaming



# Example: Handling Multiple Events

```
prompt = "Explain how a neural network learns concisely for a child."

with client.responses.create(model="gpt-5.4-mini", input=prompt, stream=True) as stream:
    for event in stream:
        if event.type == "response.created":
            print("Response started...\n")

        elif event.type == "response.output_text.done":
            print("\n\n--- Text block finished ---\n")

        elif event.type == "response.completed":
            print(f"\nFull response:\n{current_text}")
```

# Example: Handling Multiple Events

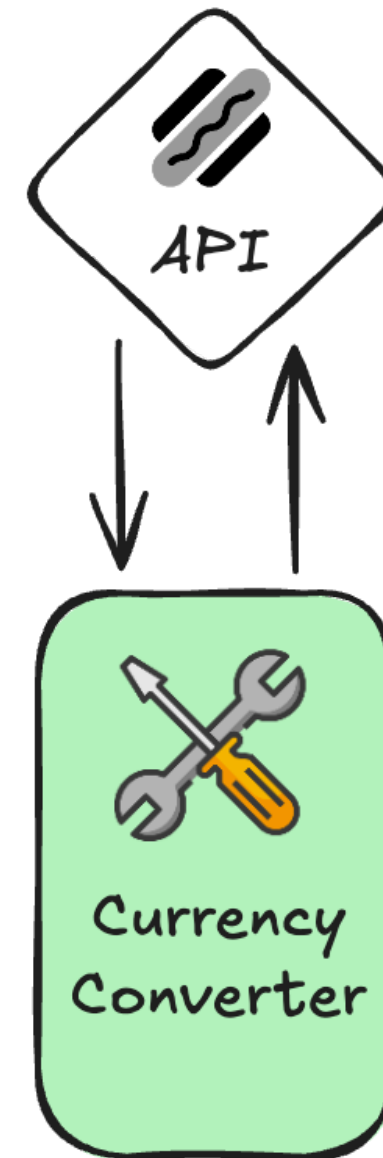


# Example: Streaming Tool Events

`convert_currency() -> str:`

- `date_time: str`
- `from_timezone: str`
- `to_timezone: str`

```
tools = [  
    {  
        "type": "function",  
        "name": "convert_currency",  
        ...  
    }  
]
```



# Example: Streaming Tool Events

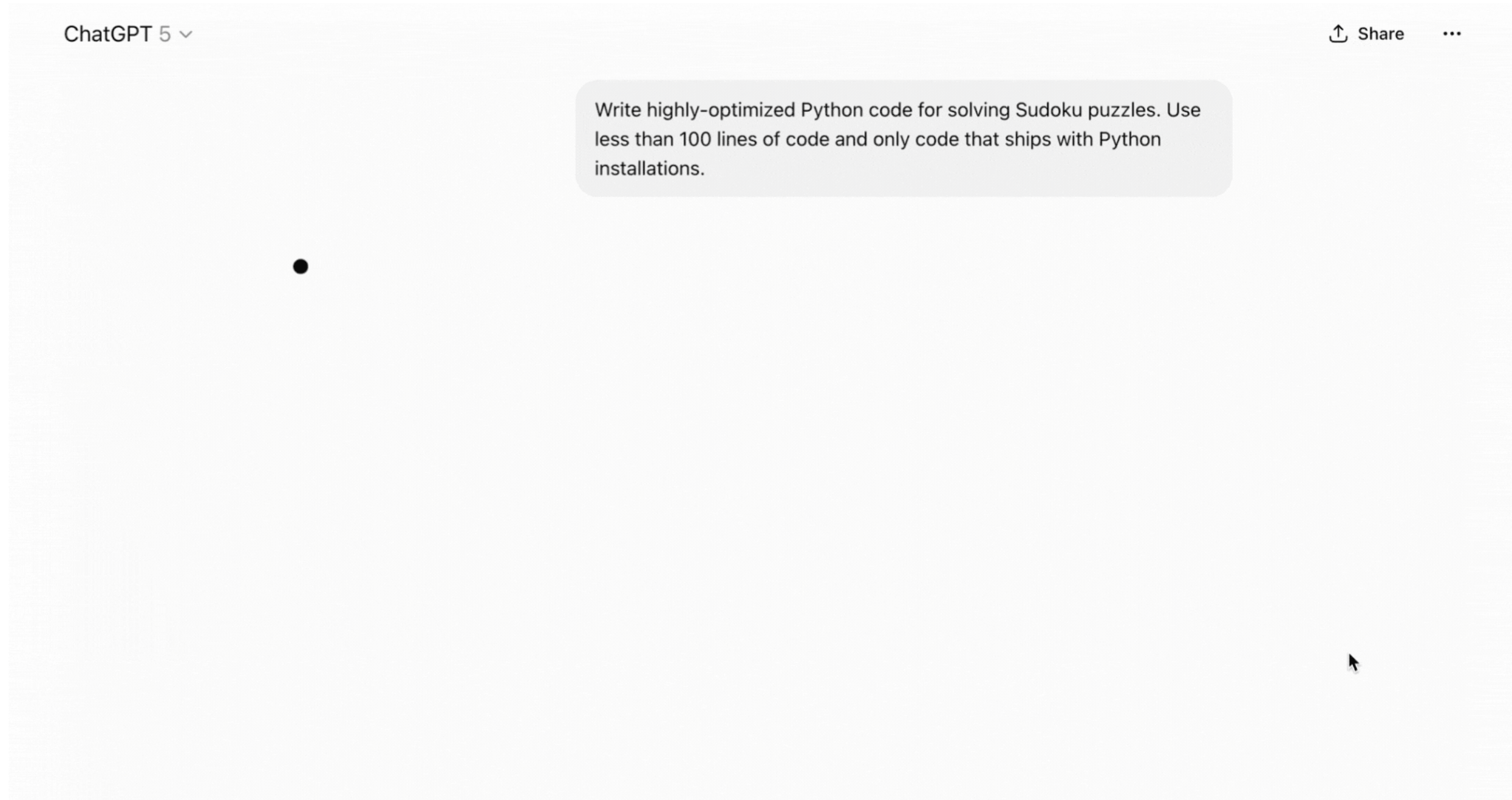
```
prompt = "How much is 120 euros in British pounds using the current exchange rate?"

with client.responses.create(model="gpt-5.4-mini", input=prompt, tools=tools, stream=True) as stream:
    current_args = ""
    for event in stream:
        if event.type == "response.function_call_arguments.delta":
            current_args += event.delta
            print("Streaming args:", current_args)
        elif event.type == "response.function_call_arguments.done":
            print("\nFinal arguments:", event.arguments)
        elif event.type == "response.completed":
            print("\n--- Completed ---")
```

# Example: Streaming Tool Events



# Summary



<sup>1</sup> Semantic Events being used in ChatGPT

# Let's practice!

WORKING WITH THE OPENAI RESPONSES API

# Input Flexibility and Multimodality

WORKING WITH THE OPENAI RESPONSES API



**James Chapman**

AI Curriculum Manager, DataCamp

# Input Flexibility

## Prompts as Strings

```
response = client.responses.create(  
    model="gpt-5.4-mini"  
    input="How old is the Earth?",  
    instructions="Answer concisely."  
)
```

- `input` → user inputs
- `instructions` → behavior + requirements

## Role-Based Prompts

```
response = client.responses.create(  
    model="gpt-5.4-mini"  
    input=[  
        {"role": "system",  
         "content": "Answer concisely."},  
        {"role": "user",  
         "content": "How old is the Earth?"}  
    ]  
)
```

# Role-Based Messages

```
response = client.responses.create(  
    model="gpt-5.4-mini"  
    input=[  
        {"role": "system", "content": "Answer concisely."},  
        {"role": "user", "content": "How old is the Sun?"},  
        {"role": "assistant", "content": "4.603 billion years"},  
        {"role": "user", "content": "How old is the Earth?"}  
    ]  
)
```

```
sys_prompt = "You are a helpful Teacher who provides concise, personalized explanations."
latest_response_id = None

while True:
    user_input = input("You: ").strip()
    if user_input.lower() == "exit":
        break

    response = client.responses.create(
        model="gpt-5.4-mini",
        instructions=sys_prompt,
        input=user_input,
        previous_response_id=latest_response_id
    )

    print(f"\nAssistant: {response.output_text}\n")
    latest_response_id = response.id
```

```
messages = [{"role": "system"},
             {"content": "You are a helpful Teacher who provides concise, personalized explanations."}]

while True:
    user_input = input("You: ").strip()
    if user_input.lower() == "exit":
        break

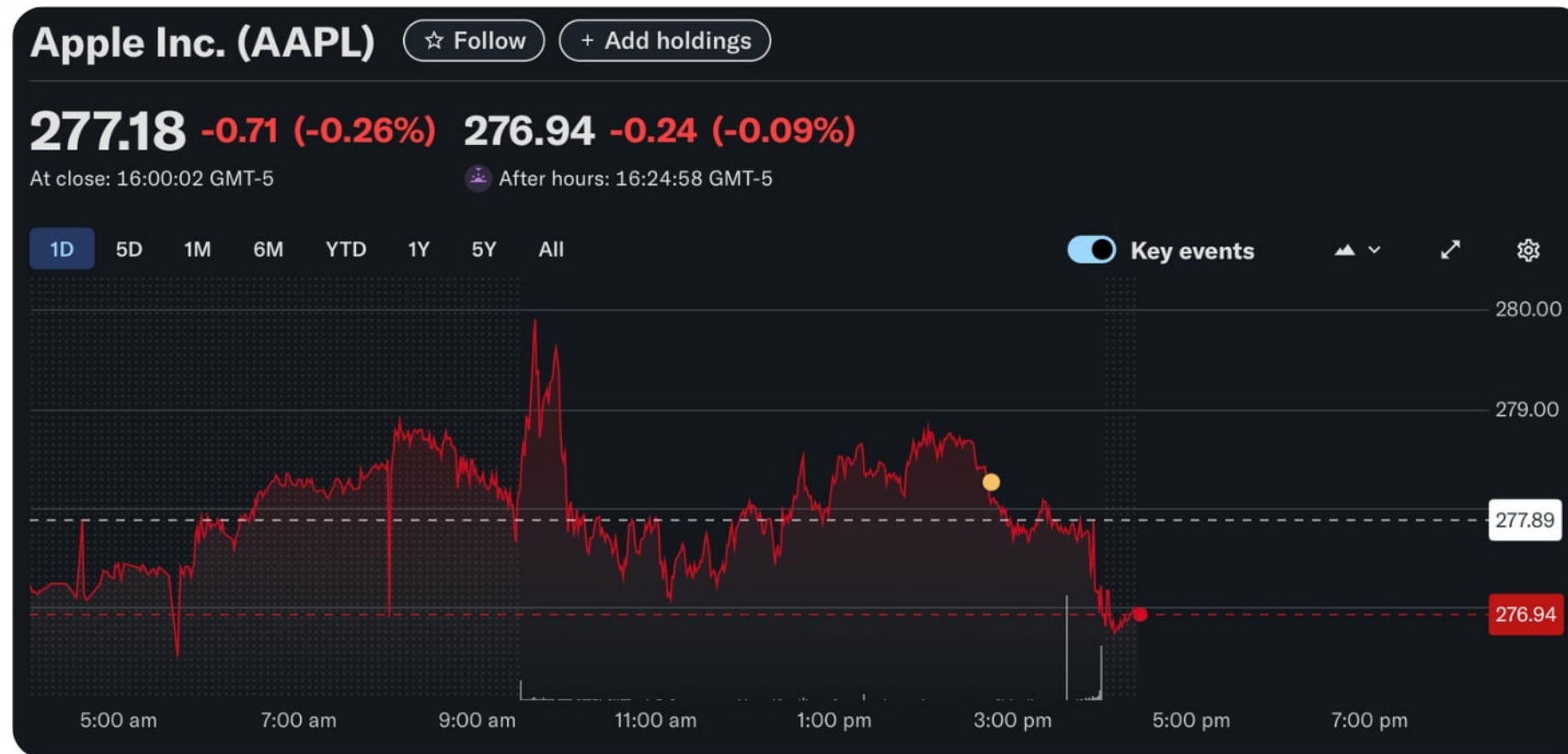
    messages.append({"role": "user", "content": user_input})

    response = client.responses.create(
        model="gpt-5.4-mini",
        input=messages
    )

    print(f"\nAssistant: {response.output_text}\n")
    messages.append({"role": "assistant", "content": response.output_text})
```

# Images in Prompts!

- Multi-Modal: interfaced through multiple content modalities



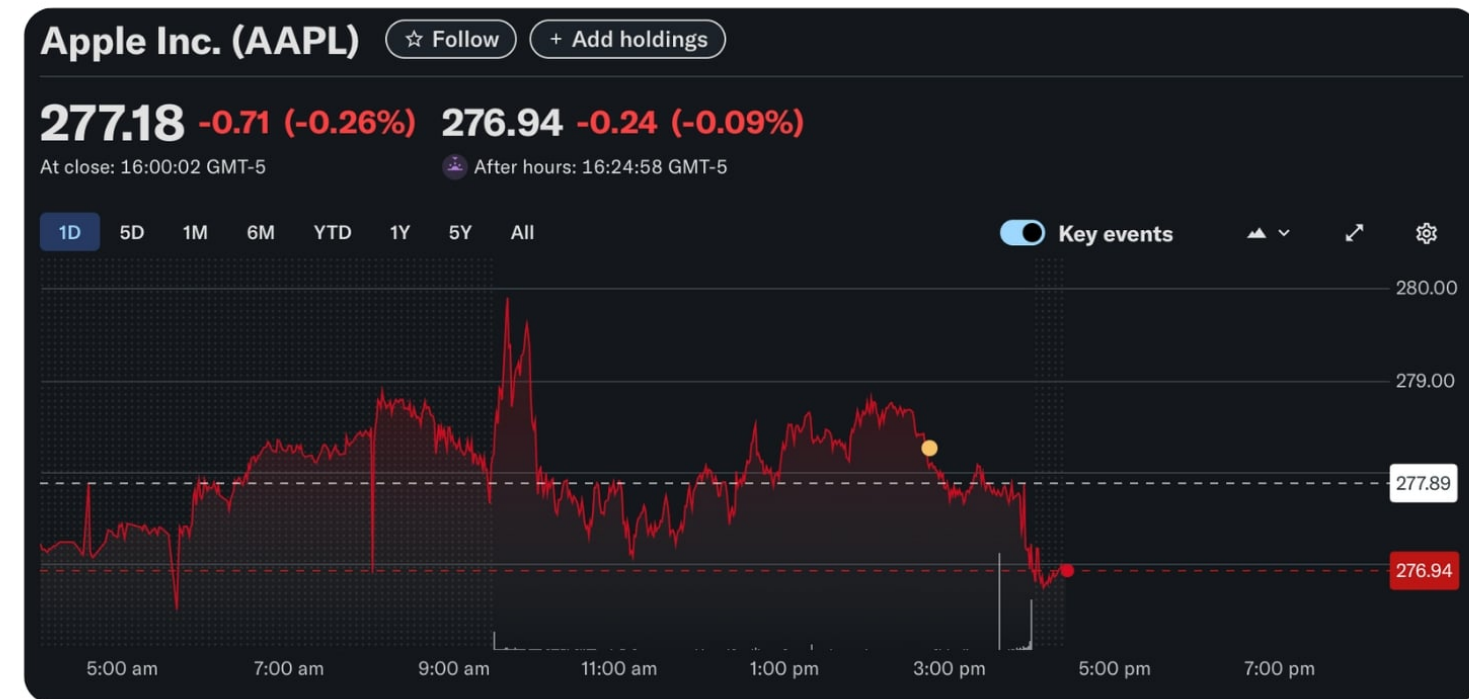
<sup>1</sup> Apple, Inc. Stock Plot from yahoo!finance

# Images URLs in Prompts

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    input=[  
        {"role": "user", "content": [  
            {"type": "input_text", "text": "Briefly interpret this stock plot."},  
            {"type": "input_image", "image_url": "https://...stock_performance.jpg"}  
        ]  
    ]  
)  
  
print(response.output_text)
```

# Images URLs in Prompts

- Summary: A mild down day - closed at 277.18, down about 0.26% from the previous close; after-hours ticked slightly lower to 276.94.



<sup>1</sup> Apple, Inc. Stock Plot from yahoo!finance

# Images from Local Files

```
import base64
# Encode binary data as ASCII text characters
with open(image_path, "rb") as f:
    image_base64 = base64.b64encode(f.read()).decode("utf-8")

response = client.responses.create(
    model="gpt-5.4-mini",
    input=[
        {"role": "user", "content": [
            {"type": "input_text", "text": "Briefly interpret this stock plot."},
            {"type": "input_image", "image_url": f"data:image/jpeg;base64,{image_base64}"}
        ]
    }
)
```

# Let's practice!

WORKING WITH THE OPENAI RESPONSES API

# Congratulations!

WORKING WITH THE OPENAI RESPONSES API



**James Chapman**

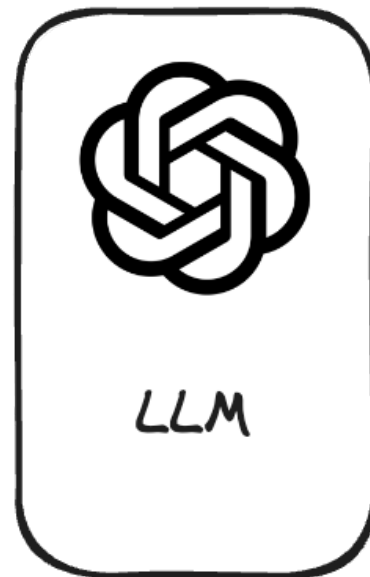
AI Curriculum Manager, DataCamp

# Chapter 1: Welcome to the Responses API

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    input="What is recursion?",  
    reasoning={"effort": "none"},  
    max_output_tokens=500  
)
```



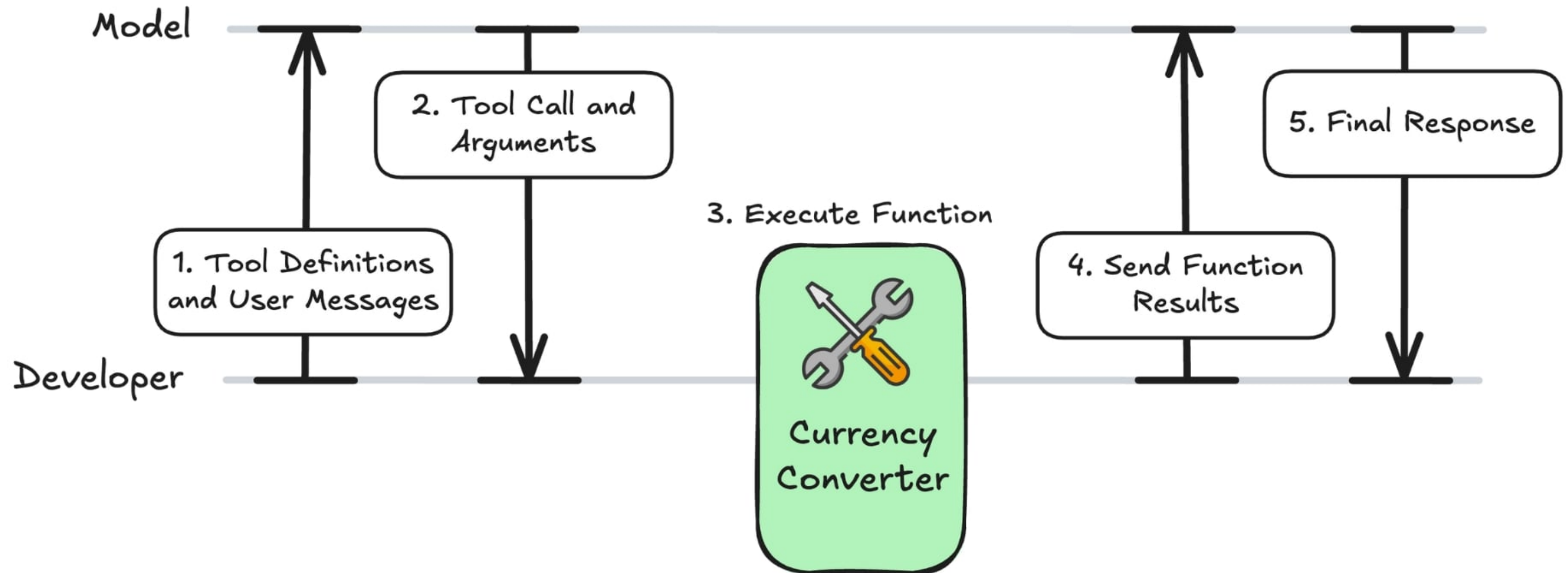
# Chapter 2: Web Search



+



# Chapter 2: Function-Calling Tools



# Chapter 3: Structured Outputs

```
from pydantic import BaseModel
```

```
class QuizResult(BaseModel):
```

```
    score: int
```

```
    passed: bool
```

```
    feedback: str
```

```
response = client.responses.parse(  
    model="gpt-5.4-mini",  
    instructions="...",  
    input="...",  
    text_format=QuizResult  
)
```

```
result = response.output_parsed
```

```
print(f"Score: {result.score}/10")
```

```
print(f"Passed: {result.passed}")
```

```
print(f"Feedback: {result.feedback}")
```

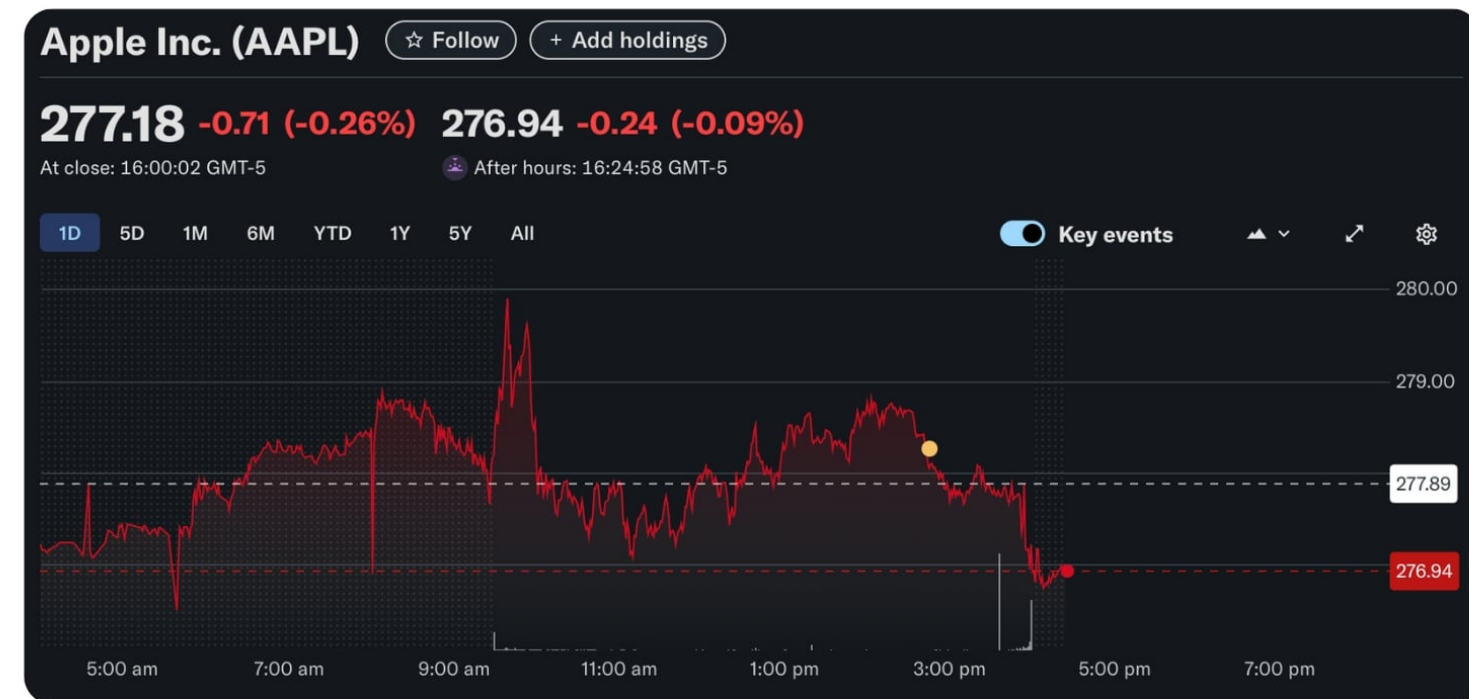
# Chapter 3: Streaming and Semantic Events

- `stream=True`

| Event Type                                          | Description                      |
|-----------------------------------------------------|----------------------------------|
| <code>response.created</code>                       | The model has started generating |
| <code>response.output_text.delta</code>             | Partial text update              |
| <code>response.output_text.done</code>              | Text block complete              |
| <code>response.function_call.arguments.delta</code> | Streaming tool arguments         |
| <code>response.completed</code>                     | The entire response is finished  |

# Chapter 3: Multimodality

- Summary: A mild down day - closed at 277.18, down about 0.26% from the previous close; after-hours ticked slightly lower to 276.94.



<sup>1</sup> Apple, Inc. Stock Plot from yahoo!finance

# Let's practice!

WORKING WITH THE OPENAI RESPONSES API